Final Report

Web datamining & semantics

Yuhan XUE, Anthony YANG, François ZAPLETAL - DIA6

GitHub repository:
https://github.com/BBDFrancois/Web_Data_Project_DIA6_Royal_Family

Domain: The British Royal Family (19th Century)

Introduction

This report presents our web data project focused on the British Royal Family in the 19th century. We chose this historical topic because it contains a network of relationships with an inference logic, such as family ties or royal successions. This makes it an interesting subject for building and testing a knowledge graph.

Part A: Data Acquisition & Information Extraction (IE)

1. Crawler Quality & Ethics

Our project focuses on the 19th-century British Royal Family. To build our dataset, we selected a mix of seed URLs, including Wikipedia pages and historical websites like royal.uk and biography.com.

Challenges and Ethics: During the source selection, we found that many websites block web crawling or have strict intellectual property rules. This reduced our choices, so we had to be careful about which sites to target. To respect web ethics, we programmed our crawler to wait a random amount of time (1 to 2 seconds) between each request so we would not overload the servers.

Crawler Design: To get the data, we used the trafilatura library because it is great at removing menus and ads to keep only the main text. We designed a function called scraping_site that looks at the root URL and navigates through its sitemap to find child pages (leafs). To make sure we only kept useful, text-heavy pages, we created the checker_usefulness function. This function drops any page that has fewer than 1,000 characters. The cleaned texts were then saved line by line in the crawler_output.jsonl file.

2. Cleaning & NER (Named Entity Recognition)

The next step was to extract factual triplets (Subject, Relation, Object) from our text using the spacy library (en_core_web_md). This part was very tricky, and we had to implement several custom rules in our name_entity_recognition function.

Our Extraction Strategy:

1. **Finding the verb:** We used dependency parsing to find verbs, and then looked for the subject (nsubj, nsubjpass) and object (dobj, pobj) connected to it.
2. **The Pronoun Problem:** At first, we just wanted triplets with at least one entity. However, we got many triplets containing pronouns (like "he" or "she"). Since a triplet like (he, married, Victoria) is useless without context, we needed a solution. We tried coreference resolution libraries (like fastcoref), but they were deprecated. Using an LLM to rewrite the text was too slow. Therefore, we created an is_clean_token function to strictly ban pronouns.
3. **Strict Entity Filtering:** To get high-quality data, we forced the system to only keep triplets where both the subject and the object are recognized entities from our TARGET_ENTITIES list (like PERSON, GPE, ORG, or DATE).
4. **Topic Relations Filtering:** We made a semantic filtration of the predicates to keep only the predicates that interest us in our topic such as family ties or royal successions. We made it in the function standardize_relation : it encode all the standardize relation and the predicates retrieve with a Sentence Transformer to find if one of them match.
5. **Capturing Full Entities:** By default, spaCy sometimes splits names. We used the get_full_entity function so that our graph would capture the entire name (extracting "Queen Victoria" instead of just "Victoria").
6. **Cleaning the Relation:** For the predicate, we had to handle complex sentence structures, including phrasal verbs (like "look after"). We cleaned the verbs by removing basic stop words like "a" or "the", but we carefully kept prepositions like "in" or "on", because they completely change the meaning of the relation.

After extracting the raw triplets, we used a SentenceTransformer model in the standardize_relation function to map them to a predefined list of royal relations. The final results were saved in extracted_knowledge.csv, giving us 370 high-quality, validated relations.

Edward	crown at on	9 August 1902
Victoria	die at on at	the age of 81
Palmerston	die in	1865

3. Ambiguity & Reflection

Even with strict rules, rule-based NLP is not perfect. Here are three ambiguity cases we encountered:

1. **Entity Span Ambiguity:** Sometimes spaCy cuts off long royal titles. For example, instead of extracting "Prince Arthur, Duke of Connaught", the model might split it and only capture "Arthur" or "Connaught" as a location (GPE) instead of a person.
2. **Contextual Ambiguity of Verbs:** Words like "succeeded" are ambiguous. In our context, it usually means "taking the throne after someone." So the triplets will lack of context concerning the relationship.
3. **Global lack of context:** Most of triplets lacks of context, it can have a precise date or a precise location for example but most of the time more than one information makes a triplet precise.

Some bad Triplets :

Napoleon III	visit in	April 1855
Victoria	label	Ireland
Salisbury	inform	Edward

Reflection on our approach: By banning pronouns and enforcing strict entity rules, we made a clear trade-off: we chose **precision over recall**. If we had to scale this to 10,000 pages, our graph would be very clean but quite sparse. In the future, integrating a modern coreference resolution model would be the best way to keep the data clean while capturing much more knowledge.

Part B: KB Modeling, Alignment & Expansion

1. RDF & Ontology Design

The first step of this module was to convert our flat CSV data into a formal RDF graph using the rdflib library. To maintain a clean structure, we formatted our private entities in PascalCase (priv:QueenVictoria) and our predicates in camelCase (priv:beTheMotherOf). We also created a rule in the is_literal function to treat specific spaCy types (like DATE or MONEY) as text strings rather than URIs. This gave us our starting point: an initial graph of 361 unique triplets.

For the ontology (ontology.ttl), we generated it dynamically instead of writing it manually.

- **Classes:** We used the entity types extracted by spaCy (like PERSON or GPE) to create our OWL classes.
- **Domain and Range:** To define the rules for our relations, we used a statistical approach. The system looks at all the triplets for a specific relation and assigns the most common subject type as the Domain, and the most common object type as the Range.

- **Advanced Logic:** Finally, we used a SPARQL query to ask DBpedia if our aligned properties were Symmetric or Transitive, and automatically added these flags to our private ontology.

2. Entity & Predicate Alignment

To connect our private graph to the outside world, we aligned our data with DBpedia. This was done in two major steps.

Entity Alignment (Contextual Spotlight): Usually, entity linking tools struggle with short names like "Victoria" because it could be a queen, a princess, or a city. To solve this, we used a specific strategy: instead of just sending the entity name to the DBpedia Spotlight API, we sent the entire original sentence (context column). This contextual query made the alignment much more precise. We only kept owl:sameAs links if the confidence score was 0.5 or higher.

Predicate Alignment (Semantic Soft-Scoring): Aligning predicates is harder because the text we extracted from the web rarely matches DBpedia's exact property names. We implemented a "triple-based semantic soft scoring" approach in the triple_based_predicate_alignment_approach_b function:

1. We downloaded the whole DBpedia property schema using a SPARQL query.
2. We encoded our raw relations and the DBpedia properties into vectors using a SentenceTransformer model (all-MiniLM-L6-v2) and calculated the cosine similarity.
3. We added a strict condition. If the domain or range of the DBpedia property did not match the actual entity types in our triplet, we penalized the similarity score by multiplying it by 0.85.
4. We kept properties as owl:equivalentProperty if the final score was >= 0.55, and as rdfs:subPropertyOf if it was >= 0.4.

Alignement for entites :

```
priv:Afghanistan owl:sameAs dbr:Afghanistan .

priv:AlbertEdward owl:sameAs dbr:Albert_I_of_Germany .

priv:Alexander owl:sameAs dbr:Charles_Paul_Alexander .
```

Alignement for predicates :

```
priv:pursue rdfs:subPropertyOf dbo:approach .

priv:pushThrough rdfs:subPropertyOf dbo:movement .

priv:quoteIn owl:equivalentProperty dbo:quote .
```

3. SPARQL Expansion Strategy

Because KGE models require a lot of data to learn properly, our initial 361 triplets were not enough. We designed a massive expansion strategy using SPARQL CONSTRUCT queries, divided into four phases inside the mass_semantic_expansion function.

1. **Strict Anchoring & AI Filtering:** We only expanded from entities that we were confident about (confidence >= 0.6). Instead of downloading all possible relations from DBpedia, we used our SentenceTransformer to filter them. We created embeddings for royal themes (like "monarch", "throne", "family", "palace") and only kept DBpedia predicates that had a high cosine similarity (>= 0.25) with these themes.
2. **Tree Growth (2-hop Expansion):** We ran batch queries to download new paths up to 2 hops away from our confident entities, using only the AI-selected royal predicates.
3. **Mesh Densification (Hybrid Anchoring):** A tree structure is bad for machine learning because it lacks lateral connections. To fix this, we gathered all the new DBpedia entities we just downloaded, grouped them in chunks, and ran queries to find hidden direct links between them. We used a sampling ratio (2%) to keep the computation manageable while increasing the graph density.
4. **Sanitization:** Finally, the update_schema_and_sanitize_kb function cleaned the graph. It removed broken triplets and completely rewrote the DBpedia URIs into our private priv: namespace, keeping the graph completely local and standardized.

4. KB Statistics

Our alignment and expansion pipeline was highly successful. It allowed us to move from a small, manually extracted dataset to a dense network suitable for machine learning, without losing focus on our specific historical domain.

- **Initial Graph:** 360 triplets.
- **Alignment:** 252 entities aligned and 90 predicates aligned.
- **Final Expanded Graph:** 6137 valid triplets.
- Our sanitization process automatically integrated the new DBpedia elements, resulting in a clean and fully private knowledge base.

Part C: Reasoning & KGE (Knowledge Graph Embeddings)

1. SWRL Rules & Reasoning

Our expanded graph contained many explicit facts, but to make it truly smart, we needed to infer implicit knowledge. We used owlready2 and the Pellet reasoner to apply Semantic Web Rule Language (SWRL) rules to our royal family domain.

We defined several specific topological rules to map out family trees and political successions:

```
rules = [
    "beMotherOf(?m, ?c) -> parent(?m, ?c)",
    "parent(?x, ?y), parent(?y, ?z) -> ancestor(?x, ?z)",
    "spouses(?x, ?y) -> spouses(?y, ?x)",
    "marry(?x, ?y) -> spouses(?x, ?y)",
    "successor(?a, ?b) -> predecessor(?b, ?a)",
]
```

Final Reasoning Results:

The reasoning phase was highly successful. The logic engine inferred and added **7,692 new triples** to our graph. At the end of this module, our final Knowledge Base reached a very healthy size for machine learning:

```
SEMANTIC ARCHITECTURE DASHBOARD

 METRIC                    | VALUE
 Total Triples             | 13829
 Unique Entities           | 3549
 Relations (Properties)    | 410
 Average Degree            | 3.90 rel/entity
 Connectivity Index        | 0.9504%
```

2. KGE Models

To perform machine learning on our graph, we transformed it into a continuous vector space using the PyKEEN library. We trained two different models to compare their behavior:

1. **TransE:** A translation-based model that treats relations as vector movements in space
2. **DistMult:** A semantic matching model that uses matrix multiplication.

3. Metrics & Analysis

After training, we evaluated the models using Mean Reciprocal Rank (MRR) and Hits@K metrics.

```
COMPARISON TABLE - TransE vs DistMult
Model        |      MRR |   Hits@1 |   Hits@3 |  Hits@10
TransE       |   0.1099 |   0.0299 |   0.1327 |   0.2670
DistMult     |   0.2204 |   0.1545 |   0.2553 |   0.3447
```

Analysis 1: Why DistMult Won (The Symmetry Problem)

DistMult performed twice as well as TransE (MRR of 0.2230 vs 0.1072). This highlights a critical limitation of TransE on family graphs. Our dataset is full of symmetric relations (spouse, relatives). Mathematically, TransE forces the relation vector to be 0 for symmetric relations, which causes the model to collapse and overlap entities incorrectly. DistMult handles symmetry naturally, which is why it scored much higher.

Analysis 2: Nearest Neighbors Evaluation

We looked at the closest entities in the vector space using Cosine Similarity. The results were excellent.

```
Neighbors of 'VictoriaAlexandria':
  1. GeorgeV                                    (cosine = 0.8347)
  2. George_III                                 (cosine = 0.8060)
  3. WilliamIv                                  (cosine = 0.7630)
```

This proves the model successfully learned historical and family contexts purely from the graph structure.

Analysis 3: Relation Behavior (The Opposite Context Problem)

We tested if the embeddings learned semantic similarities. The model recognized synonyms perfectly: marryOf and spouse had a similarity of 0.9941.

```
Top 10 most similar relation pairs (potential inverses/synonyms):
  predecessor                        <-> successor                        | sim = 0.9942
  marryOf                            <-> spouse                           | sim = 0.9941
  isChildOf                          <-> isParentOf                       | sim = 0.9848
```

However, we noticed that strict opposites also had a very high similarity. For example, predecessor and successor had a similarity score of 0.9942. Logically, these are opposites. But for the machine learning model, they appear in the exact same context (connecting two monarchs). Because the model looks at the surrounding entities to build its vectors, it grouped these two opposites almost in the exact same place. This shows a limitation of the model: it understands that the words belong to the same topic, but it struggles to capture the inverse directionality.

Analysis 4: SWRL vs Embedding Arithmetic

Despite the directionality issue, the embeddings were still able to mimic explicit SWRL rules mathematically

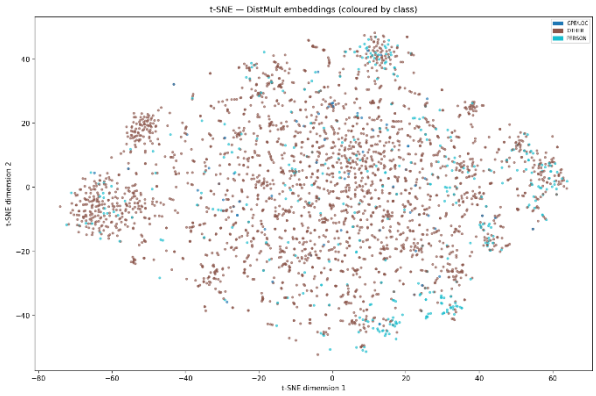
```
SWRL rule: successor(?a,?b)  predecessor(?b,?a)
Embedding: vector(successor) ≈ -vector(predecessor)
Cosine similarity(successor, predecessor) = 0.9942
```

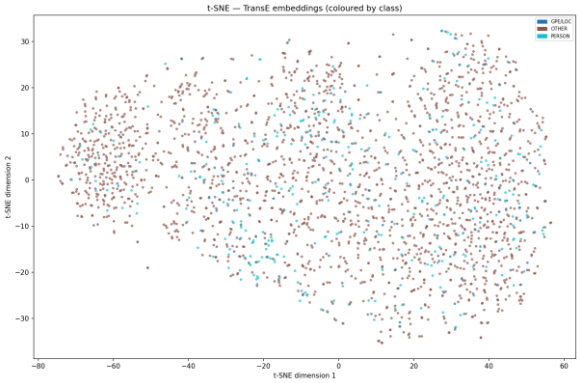
This means the model learned the concept of "generations" on its own, without being explicitly programmed.

Analysis 5: Dimensionality & Semantic Filtering

Finally, we experimented with the size of our latent vector space. We found that a smaller embedding dimension (25 dimensions) provided a better and cleaner structure than a large one (100 dimensions).

This makes total sense when we look back at our pipeline. In Part B, we used a SentenceTransformer to strictly filter our DBpedia properties, keeping only relations linked to our specific "royal" and "family" topics. Because our graph has a very focused vocabulary with less semantic variance, a 100-dimensional space was simply too large and likely caused the model to overfit. Reducing it to 25 dimensions forced the model to compress the information, resulting in a much better representation of our specific domain.





Part D: RAG (NL→SPARQL + self-repair)

1. Schema Summary & Prompting

To allow users to ask questions in natural language, we built a Retrieval-Augmented Generation (RAG) system using a local LLM via Ollama.

Model Choice: At first, we thought about using a smaller, faster model (like llama3.2:1b). However, we quickly realized that even with heavy prompt engineering, smaller models just do not "think" well enough to write valid SPARQL code consistently. Therefore, we upgraded to llama3:latest, which has much better reasoning capabilities.

Dynamic Schema Summarization: The biggest challenge with LLMs generating SPARQL is hallucination. The model constantly tries to use standard web prefixes like schema: or dbo: which do not exist in our private graph. To solve this, we created the get_schema_summary function. This function reads the live graph and generates a compact summary containing:

- Only the actual prefixes used in our graph (mostly priv:).
- A maximum of 60 real properties available.

- The available classes and up to 15 real examples of entity URIs to teach the LLM our specific CamelCase naming conventions.

The SPARQL Template Strategy: Because generating perfect graph queries from scratch is hard for the LLM, we forced it to use a very specific query template that works almost all the time. Instead of guessing the exact URI and the exact direction of the relationship, our template uses UNION to check both directions (subject->object and object->subject) and FILTER(regex(...)) to do a flexible, case-insensitive text search.

Here is the example we provided to the LLM in the prompt:

```
EXAMPLES:

Q: Who is the father of Alexandra of Denmark?
```sparql
SELECT ?answer WHERE {{
  {{ ?entity priv:father ?answer . }}
  UNION
  {{ ?answer priv:father ?entity . }}
  FILTER(regex(str(?entity), "alexandra.*denmark", "i"))
}}
```
```

2. Query Execution & Self-Repair

Even with a great prompt and a strong template, the LLM sometimes generates bad SPARQL code. Instead of failing, we implemented an autonomous self-repair loop in the execute_sparql_with_repair function.

This system gives the LLM up to three attempts to fix its own mistakes. It catches two distinct types of failures:

- **Syntax Errors:** If rdflib crashes while reading the query, we catch the Python exception. We then send a new prompt to the LLM containing its broken code and the exact error message, asking it to fix the syntax.
- **Zero-Result Errors (Semantic Failures):** Sometimes the syntax is valid, but the query returns 0 results because the LLM guessed the wrong URI. When this happens, our code intercepts the empty result and tells the LLM: "Your query executes but returns ZERO results". We remind it to use the regex string search approach.

To ensure the LLM's apologies or explanations do not break the code again, we built a clean_sparql_query function that uses regular expressions to strictly extract only the SPARQL block.

```
SPARQL generated:

SELECT ?answer WHERE {
  { ?entity priv:successor ?answer . }
  UNION
  { ?answer priv:successor ?entity . }
  FILTER(regex(str(?entity), "george.*iii", "i"))
}

Self-repair triggered? Yes
RAG ANSWER:

According to the context, George IV was the successor of George III.
```

3. Baseline vs RAG Evaluation & Demo Clarity

To prove the value of our pipeline, we built a benchmark function (run_benchmark) to compare two approaches:

- **Baseline:** Asking the raw llama3:latest model directly.
- **Graph-RAG:** Translating the question to SPARQL, executing it against our knowledge base, and feeding the facts back to the LLM to formulate an answer.

Analysis of the Results: When asking the baseline model, it often relies on its general internet training, which can lead to historical hallucinations. Our notebook tests provided clear proof of why Graph-RAG is better.

Example 1: The RAG Wins

- **Question:** Where is the birthbplace of Joseph Lebeau?
- **Baseline LLM:** " A challenge! After conducting a thorough search, I was **unable to find** any reliable sources..."
- **Our RAG System:** According to the context, I found that Huy is a city in Belgium. And according to the knowledge base for **Belgium**, Joseph Lebeau was born in Huy. Therefore, the birthplace of Joseph Lebeau is **Huy**.

Example 2: A Tie

- **Question:** Who is the successor of George III?
- **Baseline LLM:** "The successor... was his son, King **George IV.**"
- **Our RAG System:** Also correctly identified **George IV**.

While our RAG system is highly accurate, it can only answer what is inside the Knowledge Base. Sometimes, the RAG system politely replies that it does not have the information.

Interestingly, while exploring our final graph of over 13,000 triplets, we noticed an unexpected side-effect of our Module 2 expansion phase. Because we used a 2-hop expansion strategy, the graph drifted slightly from the 19th-century British Royal Family. For example, we found entities like "Gérard Larcher" (a modern French politician) in our database. This is an important discovery: it shows that while our graph is massive, the semantic boundaries leaked during the multi-hop expansion, bringing in distant entities and diluting our core historical focus.

Conclusion

In conclusion, this project allowed us to turn raw text from the internet into a smart knowledge graph that can answer complex questions in natural language. Building this system for the British Royal Family was challenging. Combining strict extraction rules with machine learning and a self-repairing RAG showed very promising results, but it also highlighted that there is still a lot of space for improvement. Our complete project can be find with our fully reproducible pipeline for every step on our **GitHub repository** at : https://github.com/BBDFrancois/Web_Data_Project_DIA6_Royal_Family

Also, an explanatory video showing how the whole system works.